

AI assist week -13.4

T. VIKAS

2303A52281

BATCH 43

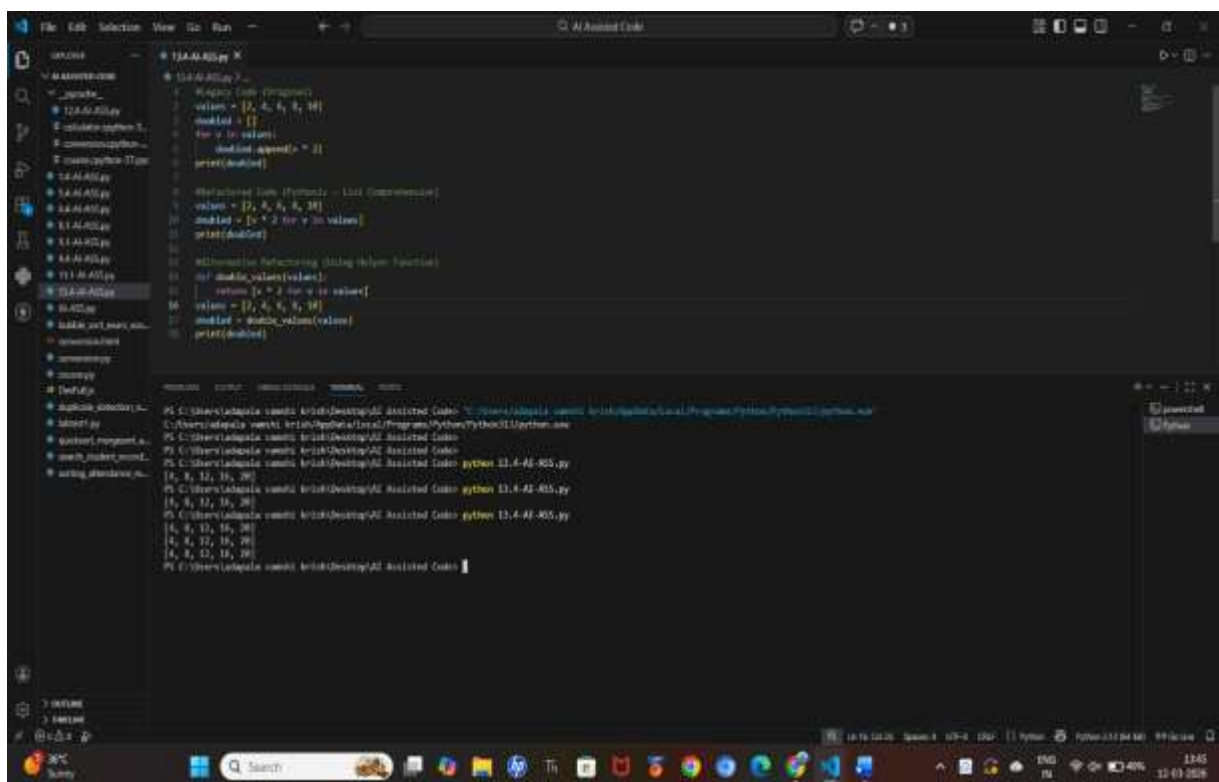
TASK-1

Scenario: (Refactoring Data Transformation Logic)

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Prompt: Refactor the following Python code to make it more Pythonic and readable using list comprehensions while keeping the same output.

Code:



```
13.4-AI-825.py
1 # Verbose Loop (Original)
2 values = [2, 4, 8, 16]
3 modified = []
4 for v in values:
5     modified.append(v * 2)
6 print(modified)
7
8 # Refactored Code (Pythonic - List Comprehension)
9 values = [2, 4, 8, 16]
10 modified = [v * 2 for v in values]
11 print(modified)
12
13 # Alternative Refactoring (Using Helper Function)
14 def double_values(values):
15     return [v * 2 for v in values]
16 values = [2, 4, 8, 16]
17 modified = double_values(values)
18 print(modified)
```

```
PS C:\Users\vikas\source\repos\13.4-AI-825\13.4-AI-825> python 13.4-AI-825.py
[4, 8, 16, 32]
PS C:\Users\vikas\source\repos\13.4-AI-825\13.4-AI-825> python 13.4-AI-825.py
[4, 8, 16, 32]
PS C:\Users\vikas\source\repos\13.4-AI-825\13.4-AI-825> python 13.4-AI-825.py
[4, 8, 16, 32]
PS C:\Users\vikas\source\repos\13.4-AI-825\13.4-AI-825> python 13.4-AI-825.py
[4, 8, 16, 32]
```

Observation:

The AI suggested using list comprehension to replace the explicit loop.

This reduced the number of lines, improved readability, and made the code more Pythonic while maintaining the same output.

TASK-2

Scenario: (Improving Text Processing Code Readability)

You are working on a log message generator that builds messages word by word.

Prompt:

Improve the following Python code that constructs a sentence using repeated string concatenation. Suggest a more efficient and readable approach.

Code:

Observation:

The AI suggested using the dictionary `get()` method to safely retrieve values.

This simplifies the code and prevents errors when a key is missing while keeping the same behavior.

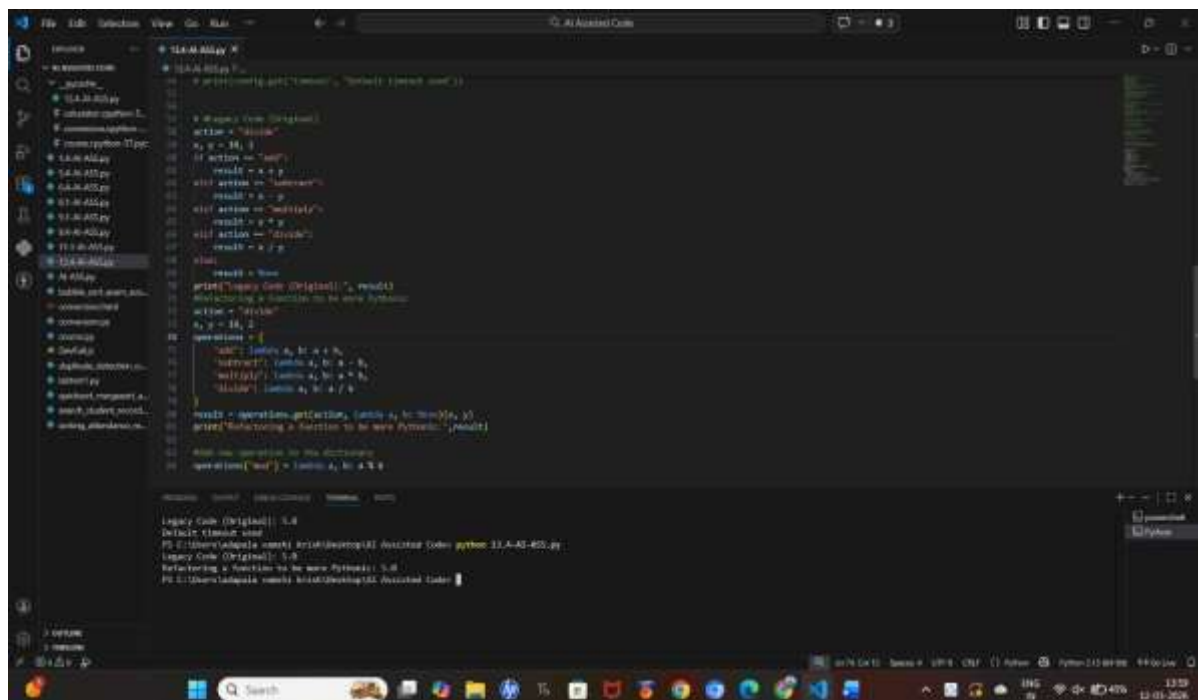
TASK-4

Scenario: (Refactoring Conditional Logic for Scalability)

You are enhancing a utility module that performs different operations based on user input.

Prompt: Refactor the following Python code that uses multiple `if-elif` conditions. Suggest a cleaner and scalable approach using mapping techniques.

Code:



```
def main():
    """Main function to handle user input and perform operations."""
    # Get user input
    action = input("Enter an action (add, subtract, multiply, divide, exit): ")

    # Perform the operation based on the action
    if action == "add":
        result = add(10, 5)
    elif action == "subtract":
        result = subtract(10, 5)
    elif action == "multiply":
        result = multiply(10, 5)
    elif action == "divide":
        result = divide(10, 5)
    else:
        result = None

    print(f"Legacy Code (Original): {result}")
    print(f"Refactoring a function to be more Pythonic")

    # Refactored code using a dictionary
    operations = {
        "add": lambda a, b: a + b,
        "subtract": lambda a, b: a - b,
        "multiply": lambda a, b: a * b,
        "divide": lambda a, b: a / b
    }

    result = operations.get(action, lambda a, b: None)(10, 5)
    print(f"Refactoring a function to be more Pythonic: {result}")

    # Add new operation to the dictionary
    operations["mod"] = lambda a, b: a % b

# Legacy Code (Original): 5.0
# Default timeout: 300s
# PS C:\Users\ladipala> python 31-A-40-405.py
# Legacy Code (Original): 5.0
# Refactoring a function to be more Pythonic: 5.0
# PS C:\Users\ladipala>
```

Observation:

The AI recommended replacing the multiple **if-elif conditions** with a **dictionary mapping of operations**.

This makes the code easier to maintain, scalable, and cleaner when adding new operations.

TASK-5

Scenario: (Simplifying Search Logic in Collections)

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Prompt: Refactor the following Python code that searches for an item in a list using a loop. Suggest a more concise and readable Pythonic approach.

Code:

```
File Edit Selection View Go Run AI Assistant Code
T3-AI-800.py
1 # Check if 'Short Version' is available in the 'Inventory' list
2 # Using 'in' operator to check for item existence
3 # Expected Output: Item Available
4
5 # Inventory list
6 inventory = ['Tool', 'Screwdriver', 'Wrench', 'Hammer']
7
8 # Check if 'Short Version' is in the inventory
9 if 'Short Version' in inventory:
10     found = True
11 else:
12     found = False
13
14 # Print the result
15 print(f'Inventory Check (Using "in" operator): {"Item Available" if found else "Item Not Available"}')
16
17 # Refactored Code (Using "in" operator)
18 inventory = ['Tool', 'Screwdriver', 'Wrench', 'Hammer']
19 found = 'Short Version' in inventory
20 print(f'Refactored Code (Using "in" operator): {"Item Available" if found else "Item Not Available"}')
21
22 # Short Version Check
23 inventory = ['Tool', 'Screwdriver', 'Wrench', 'Hammer']
24 print(f'Short Version Check: {"Item Available" if "Short Version" in inventory else "Item Not Available"}')
```

```
PS C:\Users\user> python T3-AI-800.py
Inventory Check (Using "in" operator): Item Available
Refactored Code (Using "in" operator): Item Available
Short Version Check: Item Available
PS C:\Users\user>
```

Observation:

The AI suggested using the `in` operator to check if the item exists in the list.

This eliminates the manual loop, simplifies the logic, and improves readability.